



Repository Generator

NuGet-paket för automatisk kodgenerering av Repository pattern med C# Source Generator

Christian Alexandersson
Handledare: John Strand
Företag: Dekiru Solutions AB
.NET utvecklare, 200 YH-poäng
Examensarbete 20 YH-poäng
Maj 2026
Examinator: Robin Kamo



Innehållsförteckning

1. Sammanfattning och Abstract	3
2. Introduktion	3
3. Syfte och Mål	3
Syfte	3
Mål	4
4. Metod	4
5. Resultat och analys	4
Resultat	5
Analys	5
6. Slutsatser	5
7. Källförteckning	6

1. Sammanfattning

Detta examensarbete beskriver planering, genomförande och utvärdering av RepositoryGenerator, ett NuGet paket som automatiskt genererar implementationer av Repository Pattern för Entity Framework Core applikationer med hjälp av C# Source Generators och Roslyn-kompilatorn.

Projektet genomfördes som ett fristående projekt under LIA perioden hos Dekiru Solutions AB med syftet att ta bort repetitiv boilerplate kod vid implementation av Repository Pattern. Genom att dekorera ett partiellt interface och en partiell klass med attribut genereras fullständiga CRUD-implementationer (Create, Read, Update och Delete. De fyra grundläggande operationerna för att hantera data i en databas) automatiskt vid kompileringstid.

Resultatet är ett funktionellt NuGet-paket med stöd för konfigurerbara primärnycklar och automatisk DI-registrering. Arbetet gav en djup förståelse för Roslyn och hur Visual Studio och kompilator arbetar tillsammans.

2. Abstract

This thesis describes the development of RepositoryGenerator. A NuGet package that automatically generates Repository Pattern implementations for Entity Framework Core applications using C# Source Generators and the Roslyn compiler.

The project was carried out as an independent project during the LIA period at Dekiru Solutions AB. Aiming to eliminate boilerplate code when implementing the Repository Pattern. By decorating a partial interface and a partial class with custom attributes, complete CRUD implementations are generated automatically at compile time.

The result is a functional NuGet package with support for configurable primary keys and automatic DI registration. The work provided a deep understanding of the Roslyn compiler and how Visual Studio and the compiler work together.

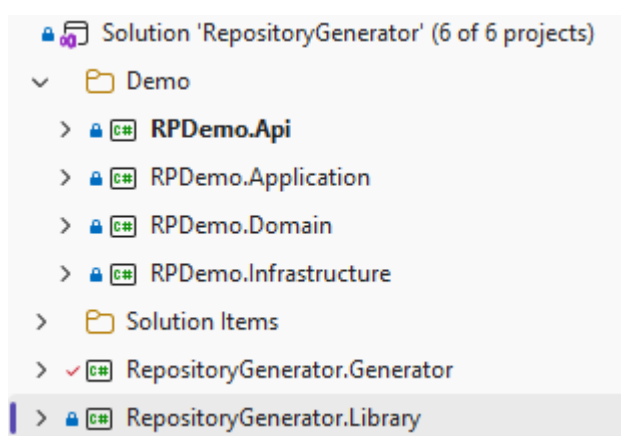
3. Introduktion

Detta examensarbete utfördes som ett delvis fristående projekt under min andra LIA-period hos Dekiru Solutions AB. På Dekiru används egenutvecklade Source Generators i det dagliga arbetet – verktyg som automatiskt skriver kod åt utvecklarna under kompileringen, det vill säga när programkoden översätts till ett körbart program. Dessa Source Generators bygger på Roslyn, Microsofts kompilatorplattform för C#, som exponerar kompilatorns analysmodell via ett API och möjliggör för program, verktyg och tillägg att läsa, analysera och generera kod som en del av kompileringsprocessen.

Examensarbetet blev därefter en personlig utmaning att förstå hur dessa verktyg fungerar på djupet, genom att bygga ett eget liknande verktyg från grunden. Projektet har utvecklats med handledning av John Strand.

Repository Pattern är ett välkänt designmönster inom mjukvaruutveckling. Enkelt förklarat är det ett sätt att organisera hur en applikation hämtar och sparar data i en databas, utan att resten av applikationen behöver känna till detaljerna kring hur det går till. Detta gör koden mer strukturerad och lättare att underhålla. Problemet är att implementationen av mönstret leder till likartad, repetitiv kod för varje entitet – det vill säga varje typ av objekt som sparas i databasen. I en applikation med tio entiteter kan det innebära tio nästan identiska klasser med samma fem metoder.

RepositoryGenerator löser detta genom att flytta ansvaret för den koden till kompilatorn. Utvecklaren anger sin intention via attribut – markeringar i koden som ger instruktioner – och generatoren producerar den färdiga implementationen automatiskt. Projektet berör flera områden inom modern .NET-utveckling: kompilorteknologi, designmönster, paketsdistribution och systemarkitektur.



4. Syfte och mål

Syfte

Syftet med projektet är att självständigt planera, utveckla och utvärdera ett NuGet-paket vid namn RepositoryGenerator. Paketet använder C# Source Generators för att automatiskt generera implementationer av Repository Pattern baserade på entiteter i en Entity Framework Core DbContext.

Paketet ska eliminera behovet av manuellt skriven, repetitiv CRUD-kod för varje entitet och i stället låta utvecklaren deklarativt märka upp sina klasser och interfaces med attribut. Generatorn producerar sedan en fullständig och korrekt implementation automatiskt vid kompileringstid. Syftet är att höja produktiviteten, minska risker för buggar i repetitiv kod och göra applikationsarkitekturen mer konsekvent och underhållbar. Projektet syftar även till att ge en kunskap om Roslyn-kompilatorn och kompileringstidsbaserad kodgenerering som teknik.

Mål

Målen med projektet är:

- Generatorn ska identifiera entiteter registrerade i en Entity Framework Core DbContext och lokalisera rätt DbSet<T>-egenskap för varje entitet via semantisk analys.
- Generatorn ska vid kompileringstid producera korrekta repository interfaces med de asynkrona metodsignaturerna GetById, GetAll, Add, Update och Delete.
- Generatorn ska producera fullständiga klassimplementationer som implementerar dessa interfaces och använder Entity Framework Core för dataåtkomst via rätt DbSet och SaveChangesAsync.
- Generatorn ska automatiskt producera extension-metoden AddGeneratedRepositories() för IServiceCollection, så att samtliga genererade repositories registreras i DI containern med ett enda anrop.
- Primärnycklar ska kunna konfigureras via namnkonvention (Id, eller EntityNameId där EntityName är entitetens namn, exempelvis PersonId för entiteten Person av typen int) eller explicit via attribut, med stöd för typerna int, long och string.
- Paketet ska demonstreras i ett fungerande demoprojekt med clean architecture-struktur (Domain, Application, infrastructure, API)

5. Metod

Arbetsprocess och planering

Projektet inleddes med en förberedelsefas där fokus låg på att bygga upp tillräcklig förståelse för tekniken innan någon kod skrevs. Detta innebar att läsa Microsofts dokumentation för Roslyn och Source Generators, studera Dekirus befintliga Source Generators för att förstå hur de var uppbyggda i praktiken, samt följa handledningsfilmer för att se hela flöden från attributdefinition till genererad kod.

När jag hade fått en förståelse för detta påbörjades implementation. Arbetet började med att definiera och förstå attributklasserna och generatorns publika API mot användaren. Därefter implementerades förmågan att läsa av och tolka dessa attribut via Roslyns semantiska modell, vilket var det tekniskt mest grundläggande steget. Med attributläsning på plats byggdes kodgenereringen succesivt ut: först interfacegenerering sedan klassimplementationer och DI-registrering. Det sista som implementerades var det flexibla primärnyckelstödet. Detta tillägg gjordes när resten av systemet fungerade och var stabilt, och ger användaren möjlighet att styra både primärnyckelns egenskapsnamn via attributet [PrimaryKeys] och dess typ via attributet [PrimaryKeyTypels<TKey>].

Tack vare handledarens löpande vägledning och tillgång till företagets befintliga Source Generatorer-implementationer kunde en grunddesign och projektstruktur etableras tidigt i projektet. Detta gav en bra grund att bygga vidare på och underlättade utveckling avsevärt. Handledaren genomförde även löpande kodgranskning, vilket bidrog till kontinuerlig förbättring av kod och struktur.

Teknikval och motivering

Projektet är implementerat i C# med .NET Standard 2.0 vilket är det lägsta ramverket som krävs för att Source Generators ska vara kompatibla i alla stödda .NET miljöer. Projektet använder Inkrementell Source Generator som introducerades i version av .NET 6 vilket ersatte den ursprungliga Source Generator som introducerades i .NET 5.

Biblioteken Microsoft.CodeAnalysis (Roslyn) och Microsoft.CodeAnalysis.CSharp som båda är en del av Roslyns kompilatorplattform, är obligatoriska för att utveckla Source Generators, det finns inga alternativa bibliotek för ändamålet. Dessa paket exponerar kompilatorns interna modell via ett väldefinierat API som gör det möjligt att analysera och inspektera C# kod programmatiskt vid kompileringstid.

Entity Framework Core valdes som ORM (Object Relational Mapping) eftersom det är en av de ORM ramverk som dominerar inom .NET och det ramverk som används på Dekiru. Avgränsningen till EF Core möjliggjorde fokus på Source Generatormekaniken snarare än att stödja andra ORM-ramverk. Eftersom Entity Framework Core primärt används mot SQL-databaser avgränsades projektet till SQL och standard CRUD-operationer, vilket gjorde mitt mål hanterbart och möjliggjorde att projektet blev klart inom tidsramen.

Visual Studio 2026 användes som primär IDE. Visual Studio har inbyggt stöd för att visa Source Generator-output direkt i Solution Explorer, vilket var ovärderligt under utvecklingen. NuGet användes för distribution av paketet, vilket är standard för .NET-bibliotek.

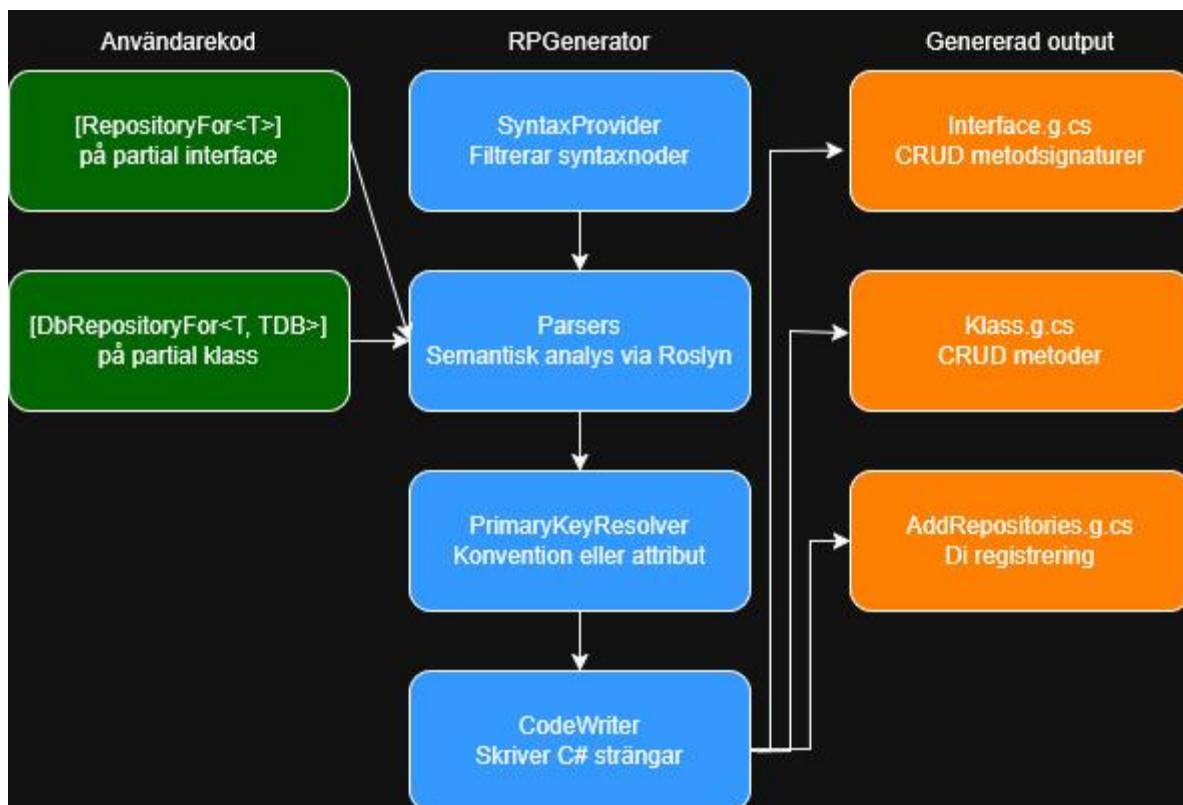
Projektstruktur och arkitektur

Lösningen är uppdelad i tre huvudprojekt. Ett medvetet val för att göra paketet användbart och underhållbart.

RepositoryGenerator.Generator. Detta är själva Source Generator-projektet. Denna innehåller IncrementalGenerator-implementationen (RPGenerator), parsers (InterfaceTargetParser, RepositoryTargetParser), primärnyckelupplösning (PrimaryKeyResolver) och kodskrivaren (CodeWriter). Detta projekt refereras enbart som kompileringsberoende och hamnar aldrig i användarens körbara applikation.

RepositoryGenerator.Library. Attribut-bibliotek som användaren refererar till vid exekvering. Innehåller de fyra attributklasserna som utgör generatorns publika API mot användaren. Separationen är medveten: användarna ska inte behöva dra in Roslyn-beroenden in i sin applikation.

RPDemo. Ett demoprojekt uppdelat i API, Application, Domain och Infrastructure-enlighet med clean architecture-principerna. Det demonstrerar hur paketet används i ett realistiskt sammanhang och fungerar som ett integrationstest för generatoren.



IncrementalGenerator API:et

Source Generators i modern .NET bygger på IncrementalGenerator-interfacet. Det inkrementella API:et är designat för IDE-prestanda: istället för att köra hela analysen vid varje tangenttryckning cache-lagras mellanresultatet och kör bara om de delar av pipeline vars indata förändrats körs om. Förändringar upptäcks genom att varje steg i pipeline jämför sin indata med tidigare körning via likvärdighetskontrollen `IEquatable<T>`, vilket kräver att modellklasserna implementerar `IEquatable<T>` med korrekt `GetHashCode` implementation, utan detta fungerar inte cache-lagringen korrekt och generatoren körs om i onödan vid varje kompilering, vilket försämrar prestandan märkbart. Detta är avgörande för att generatoren inte ska försämra skrivupplevelsen i Visual Studio.

Generatorn registrerar två separata pipelines i Initialize-metoden:

Pipeline1:

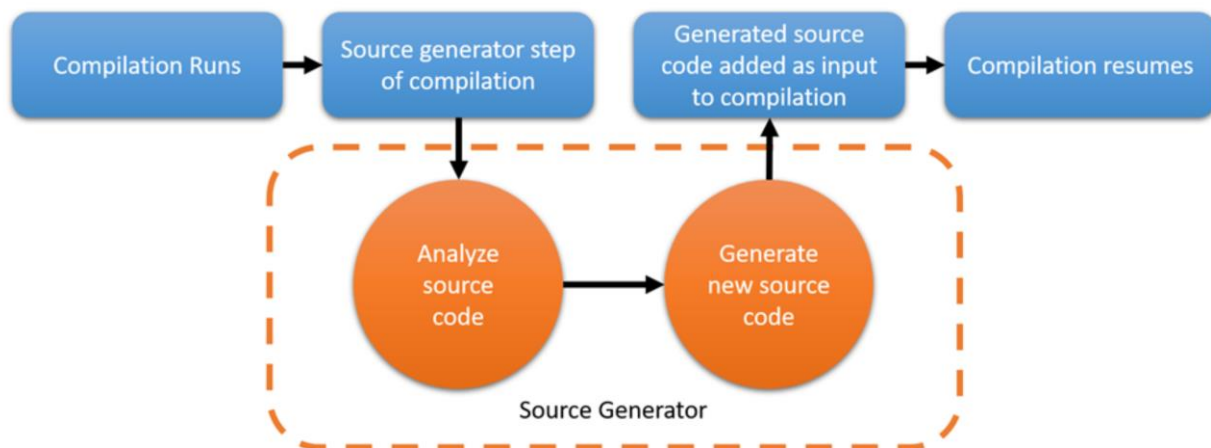
```
var interfaces = context
    .SyntaxProvider.ForAttributeWithMetadataName(
        InterfaceExtensionAttribute,
        static (_, _) => true,
        (ctx, _) => InterfaceTargetParser.TryParse(ctx)
    )
    .Collect();
```

Pipeline2:

```
var classes = context
    .SyntaxProvider.CreateSyntaxProvider(
        predicate: static (node, _) => IsClassTarget(node),
        transform: static (ctx, _) => RepositoryTargetParser.Parse(ctx)
    )
    .Collect();
```

Pipeline 1: Interface-pipelinen använder ForAttributeWithMetadataName med det fullt kvalificerade attributnamnet. Detta är det rekommenderade sättet i Roslyn's dokumentation eftersom det låter kompilatorn filtrera noder internt utan att besöka varje syntaxnod i lösningen.

Pipeline 2: Klass-pipelinen använder ett tvåstegsfilter: ett snabbt syntaxfilter (IsClassTarget) som kontrollerar att noden är av typen ClassDeclarationSyntax och att den är deklarerad med attributet DbRepositoryFor, följt av den semantiska transformeringen i RepositoryTargetParser. Uppdelningen i ett billigt syntaxfilter och ett semantiskt steg är ett designmönster som Roslynteamet rekommenderar för att hålla generatorprestandan hög.



Attributsystemet

Fyra attributklasser definierar generatorns publika API mot användaren. Designen av detta API var ett av det viktigaste arkitektoniska besluten i projektet. Attributen ska vara intuitiva att använda och ge tillräcklig information för att generera korrekt kod. De fyra attributen är:

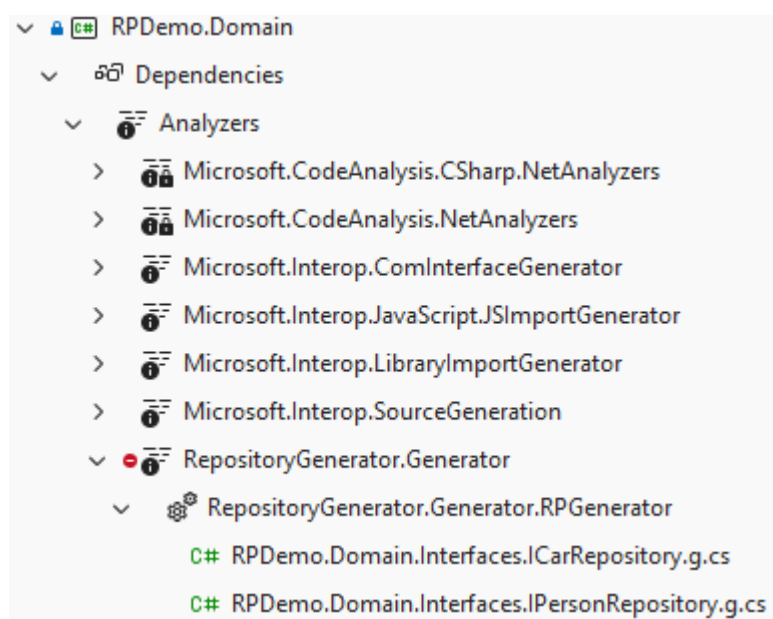
[RepositoryFor<T>] - Placeras på ett partiellt interface och anger vilken entitet T interfacet representerar ett repository för. Det utlöser kodgenerering av de fem CRUD-metodsignaturerna.

[DbRepositoryFor<T, DbContext>] - Placeras på en partiell klass och kopplar samman entiteten T med en specifik DbContext-typ. Detta skapar kodgenerering för fullständig Entity Framework Core-implementation av CRUD-metoderna.

[PrimaryKeys(string name)] - Är valfritt och används för att explicit ange primärnyckelns egenskapsnamn när konventionsbaserad lösning inte är möjlig.

[PrimaryKeyTypels<TKey>] - Är valfritt och används för att explicit ange primärnyckelns typ (int, long eller string) på interfacenivå.

Attributbiblioteket separeras medvetet från generatorprojektet. Generatorprojektet refereras som en AnalyzerReference, inte en vanligt ProjectReference, vilket innebär att det enbart körs vid kompilering och aldrig hamnar i användarens slutliga applikation. Användarna behöver bara dra in "Library" projektet, som är minimalt utan tunga Roslyn-beroenden.



Parsning och semantisk analys

InterfaceTargetParser.Parse tar emot en GeneratorAttributeSyntaxContext och extraherar namespace, interfacenamn, entitetstyp och primärnyckeltyp. Varje steg i parsningen valideras, om informationen saknas eller är inkonsekvent returneras null, vilket signalerar till generatoren att hoppa över källfilen utan att producera felaktig kod. Detta är viktigt för en bra användarupplevelse, ett fel i en Source Generator kan lätt leda till hundratals följdfe i det projekt där den används, eftersom felaktig genererad kod sprider sig genom kompileringen och gör det svårt att identifiera grundproblemet. En fel konfigurerad attributmarkering ska inte krascha bygget, den ska bara inte generera något.

RepositoryTargetParser.Parse utför en mer komplex analys. Utöver att extrahera klassens egna metadata behöver parsern lokalisera rätt DbSet<T>-egenskap i den angivna DbContext typen. Detta sker genom semantisk sökning bland DbContext-typens medlemmar:

```
var dbsetName = dbContextArgument
    .GetMembers()
    .OfType<IPropertySymbol>()
    .Where(p =>
        p.Type is INamedTypeSymbol namedType
        && SymbolEqualityComparer.Default.Equals(
            namedType.OriginalDefinition,
            dbSetSymbol
        )
        && SymbolEqualityComparer.Default.Equals(namedType.TypeArguments[0], entity)
    )
    .Select(p => p.Name)
    .FirstOrDefault();
```

Typsymboler i Roslyn är referenstyper och har därför samma begränsningar vad gäller jämförelser som andra referenstyper i .NET, två instanser kan innehållsmässigt vara identiska men en vanlig referensjämförelse (==) kommer att anse dem vara olika. Detta är särskilt problemmatiskt i Roslyn eftersom samma typ kan representeras av olika symbolobjekt i olika kompileringskontexter. Användningen av SymbolEqualityComparer.Default löser detta genom att jämföra symbolernas innehåll snarare än deras referenser.

Primärnyckelupplösning

Primärnyckelupplösningen i `PrimaryKeyResolver` följer en tvåstegsordning. Först kontrolleras om klassen dekorerats med `[PrimaryKeys]`-attributet, om det har gjorts används det angivna namnet. Annars faller systemet tillbaka på konvention baserad upplösning: generatoren söker efter en egenskap som heter `Id` eller `EntityNameId` (t.ex. `ProductId`, för en `Product` entitet) av typen `int`.

Typupplösningen för primärnyckel läser `[PrimaryKeyTypes<TKey>]`-attributet och konverterar Roslyn's `SpecialType` enum till C#-nyckelord via en switch-uttryck:

```
private static string ToTypeKeyword(AttributeData primaryKeyAttribute)
{
    var keyType = primaryKeyAttribute.AttributeClass?.TypeArguments.FirstOrDefault();

    return keyType?.SpecialType switch
    {
        SpecialType.System_Int32 => "int",
        SpecialType.System_Int64 => "long",
        SpecialType.System_String => "string",
        _ => keyType?.ToString() ?? "int",
    };
}
```

Kodgenerering

`CodeWriter` producerar de genererade C#-filerna som strängar via tre statiska metoder. Alla genererade filer namnges enligt konventionen `Namespace.ClassName.g.cs`, detta för att tydligt markera att de är autogenererade och inte kan redigeras manuellt.

Metoden `WriteRepositoryClass` genererar en partiell klass som utnyttjar C# 12:s primärkonstruktor-syntax för att ta emot en `DbContext` via dependency injection:

```
var code =
    $"
using {entityUsingName};
using {interfaceUsingNamespace};
using {dbContextUsingName};
using Microsoft.EntityFrameworkCore;

namespace {classNameNamespace}
{{
    public partial class {className}({dbContextName} context) : {interfaceName}
    {{
        public async Task<{entityName}?> GetById({primaryKeyType} id)
        {{
            return await context.{dbName}.FirstOrDefaultAsync(x => x.{primaryKey} == id);
        }}
    }}
}
```

Metoden WriteDiRegistration genererar klassen RepositoryGeneratorServiceCollectionExtension med extension-metoden AddGeneratedRepositories. Metoden samlar alla genererade interface och klasser och registrerar dem som Scoped i IServicecollection. Klassen genereras utan namespace, vilket gör den globalt tillgänglig utan using-direktiv, detta gjorde jag medvetet för att minimera friktion vid användning.

Valet att använda partiella klasser och partiella interfaces som måltyp är möjliggör för användaren att lägga till egna metoder i samma klass utan att kollidera med den genererade koden och det är ett etablerat mönster för Source Generators i .NET. Det innebär också att den genererade koden automatiskt hamnar i samma namespace som användarens klass, utan att man explicit behöver ange klassnamnet. Detta är ett etablerat mönster för Source Generator i .NET.

6. Resultat och analys

Resultat

Projektet resulterade i ett fungerande NuGet-paket som uppfyller samtliga tekniska mål. En användare av paketet behöver enbart fyra steg för att få en fullt fungerande implementation av repository pattern.

Steg 1: definiera entiteten i Domain-lagret:

```
public class Person
{
    0 references
    public int PersonId { get; set; }
    1 reference
    public long DemoPkId { get; set; }
    0 references
    public string Name { get; set; } = string.Empty;
}
```

Steg 2: dekorera interfacet i Domain-lagret:

```
[PrimaryKeyTypeIs<long>]
[RepositoryFor<Person>]
4 references
public partial interface IPersonRepository { }
```

Steg 3: dekorera klassen i Infrastructure-lagret:

```
[PrimaryKeyIs("DemoPkId")]  
[DbRepositoryFor<Person, DemoDbContext>]  
2 references  
public partial class PersonRepository : IPersonRepository { }
```

Steg 4: registrera alla repositories i DI-containern:

```
ServiceCollection services = new();  
  
services.AddGeneratedRepositories();
```

Vid kompilering genererar RepositoryGeneratoren automatiskt följande interface-implementation:

```
public partial interface IPersonRepository  
{  
    1 reference  
    Task<Person?> GetById(long id);  
    1 reference  
    Task<IReadOnlyList<Person>> GetAll();  
    1 reference  
    Task Add(Person entity);  
    1 reference  
    Task Update(Person entity);  
    1 reference  
    Task Delete(Person entity);  
}
```

Och följande klassimplementation:

```
public partial class PersonRepository(DemoDbContext context) : IPersonRepository
{
    1 reference
    public async Task<Person?> GetById(long id)
    {
        return await context.Persons.FirstOrDefaultAsync(x => x.DemoPkId == id);
    }

    1 reference
    public async Task<IReadOnlyList<Person>> GetAll()
    {
        return await context.Persons.ToListAsync();
    }

    1 reference
    public async Task Add(Person entity)
    {
        context.Persons.Add(entity);
        await context.SaveChangesAsync();
    }

    1 reference
    public async Task Delete(Person entity)
    {
        context.Persons.Remove(entity);
        await context.SaveChangesAsync();
    }

    1 reference
    public async Task Update(Person entity)
    {
        context.Persons.Update(entity);
        await context.SaveChangesAsync();
    }
}
```

Demoprojektet (RPDemo) visar paketet i en clean architecture-applikation med API, Application, Domain och Infrastructure-lager. Genererade filer syns i Visual Studio under Dependencies/Analyzers i solution explorer och kan inspekteras direkt i IDE:n, vilket underlättar felsökning och förståelse för vad generatoren producerar.

Analys

Ser jag tillbaka på projektet är jag nöjd med att samtliga mål som sattes upp i planeringen uppnåddes.

Det moment som visade sig vara tekniskt mest krävande var det inkrementella API:et, närmare bestämt kravet på `IEquatable<T>`-implementationen som beskrivs i avsnittet `IncrementalGenerator API:et`. Här hade jag Dekirus generatorer som mall och kunde se i praktiken hur det skulle implementeras.

Det finns tekniska begränsningar i den nuvarande implementationen som jag är medveten om. En av dem är att `SaveChanges`-anropet ligger inbakat i varje enskild metod, vilket innebär att användaren inte kan styra när ändringar sparas, något som kan vara problematiskt i scenarion där man vill samla flera operationer i en och samma transaktion. En annan begränsning är att det genererade metoderna inte kan skrivas över, vilket kan göra det svårt för användaren att anpassa beteendet för en specifik entitet utan att bryta mot det genererade kontraktet. Dessa är tydliga förbättringspunkter inför framtida versioner.

Projektet har gett mig betydligt mer än bara kunskap om Source Generators. Arbetet med Roslyn har gett en djup förståelse för hur faktiskt Visual studio arbetar under huven, hur kompilatorn kontinuerligt analyserar koden medan man skriver, hur den upptäcker icke körbar kod i realtid och hur IDE:ns felmarkering och IntelliSense hänger ihop med den semantiska modellen. Det är en kunskap jag tror är svår att få grepp om att faktiskt själv utveckla något.

Avgränsningarna jag satte upp innan jag började arbetet som innebar att jag använde mig av SQL, CRUD och Entity Framework Core är jag glad för. Det gav mig möjlighet att få en någorlunda komplett lösning inom min tidsram och utan att lägga allt för mycket tid utöver. Jag känner dock att jag förstår hur och vad som skulle behövas utvecklas vidare och hur man ska gå tillväga med detta. Det är enbart en fråga om tid att utföra denna utveckling.

7. Slutsatser

Arbetet med RepositoryGenerator har gett mig en djupgående förståelse för ett av de mer avancerade områdena inom modern .NET-utveckling: kompilatorbaserad kodgenerering med Roslyn. Inlärningskurvan var brant, framför allt kring IncrementalGenerator-funktionalitet samt att förstå hur Roslyns syntax API är uppbyggt med dess namngivning och struktur.

En av de största utmaningarna under projektet var felsökning och debuggning av generatoren. Till skillnad från vanlig applikationsutveckling körs generatoren inuti kompilatorn, vilket gör att vanliga debuggningverktyg inte fungerar på samma sätt. När generatoren producerade felaktig kod eller kraschade fick jag ofta svårtolkade kompileringsfel i demoprojektet snarare än i generatoren själv. Vilket gjorde det svårt att faktiskt förstå var det riktiga problemet låg. Att lära sig tolka dessa fel tog sin tid.

Något som jag tycker fungerade väl var designen av attribut-API:et, hur användaren via enkla attribut kan styra hela genereringen. En annan viktig insikt var att en Source Generator inte kan arbeta utanför det projekt som direkt använder den. Det innebär att användaren måste installera RepositoryGenerator-paketet i varje projekt där generatoren ska användas. Generatoren kan inte läsa från andra projekt även om det ligger inom samma solution. Det betyder att vid implementationen av clean architecture det att användaren måste referera till RepositoryGenerator-paketet i Domain-biblioteket och i Infrastructure-biblioteket. Det är något som är viktigt att användaren förstår.

Projektet har stärkt min förståelse för hur Dekirus interna NuGet-paket fungerar och gett mig en helt ny förståelse hur Visual Studio och kompilatorn arbetar tillsammans. Jag känner att jag kan förstå och lättare implementera andras Source Generators.

Planen är vidareutveckla RepositoryGenerator med två förbättringar: att ge användaren möjligheten att själv välja om SaveChanges ska anropas automatiskt i varje repository metod eller hanteras manuellt, samt ge användaren möjlighet att skriva över genererade metoder för att anpassa beteendet för specifika entiteter.

8. Källförteckning

- <https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/source-generators-overview>
- <https://github.com/dotnet/roslyn/blob/main/docs/features/incremental-generators.md>
- <https://www.thinktecture.com/en/net/roslyn-source-generators-introduction/>
- <https://andrewlock.net/creating-a-source-generator-part-1-creating-an-incremental-source-generator/>
- <https://specterops.io/blog/2024/10/01/dotnet-source-generators-in-2024-part-1-getting-started/?source=rss----f05f8696e3cc---4>
- <https://github.com/dotnet/roslyn/blob/main/docs/features/source-generators.cookbook.md>